Software Report
Audio360

Team #6, Six Sense
Omar Alam
Sathurshan Arulmohan
Nirmal Chaudhari
Kalp Shah
Jay Sharma

1 Revision History

Table 1: Revision History

Date	Developer(s)	Change
Mar 7, 2026	Kalp	Direction of Arrival theory and prototyping sections.

Contents

1 Revision History **i**

2 Introduction **1**

3 Direction of Arrival **1**

3.1 DoA Theory 1

3.1.1 GCC-PHAT (Generalized Cross-Correlation with Phase Transform) 1

3.1.2 MUSIC (Multiple Signal Classification) 3

3.1.3 SRP-PHAT (Steered Response Power with Phase Transform) 4

3.1.4 FRIDA (FRI-Based DOA) 4

3.2 DoA Python Prototyping 5

3.2.1 Room Simulation with PyRoomAcoustics 5

3.2.2 Test Data Generation 6

3.2.3 Algorithm Testing and Comparison 8

4 Classification **9**

List of Tables

1 Revision History i

List of Figures

1 Room layout used in PyRoomAcoustics simulation. 6

2 Microphone responses after room simulation. 7

3 Frequency-domain responses of microphone signals. 8

4 DoA estimation results: comparison of MUSIC, SRP-PHAT, and FRIDA with ground truth. 9

2 Introduction

[Explain the purpose of this document. Why it was selected for this project. —SS] [Briefly outline the structure of the document. —SS]

3 Direction of Arrival

Direction of arrival (DoA) estimation is a fundamental capability of Audio360. Given synchronized multi-channel audio from a microphone array, the system must estimate the angular direction of sound sources on a 2D horizontal plane relative to the user (FR5.2, FR5.3). This section presents the theoretical foundations of the DoA algorithms used in the project and describes the Python prototyping workflow employed to validate and compare them.

3.1 DoA Theory

The DoA problem can be stated as follows. Let $\mathbf{x}_m(t)$ denote the signal received at microphone m at time t, for $m = 1, \ldots, M$ and $M \geq 2$. A sound source at position $\mathbf{p}_s$ emits a signal $s(t)$ that propagates through the medium. Due to the finite speed of sound c, the signal arrives at each microphone with a time delay $\tau_m = \|\mathbf{p}_m - \mathbf{p}_s\|/c$, where $\mathbf{p}_m$ is the position of microphone m. The received signal is a delayed, attenuated, and possibly reverberant version of the source. Estimating the direction of arrival amounts to inferring the direction vector from the array center toward the source from the observed multi-channel signals.

The project employs four DoA algorithms: Generalized Cross-Correlation with Phase Transform (GCC-PHAT), Multiple Signal Classification (MUSIC), Steered Response Power with Phase Transform (SRP-PHAT), and FRI-based DOA (FRIDA). Each algorithm is described below.

3.1.1 GCC-PHAT (Generalized Cross-Correlation with Phase Transform)

GCC-PHAT estimates the time difference of arrival (TDOA) between pairs of microphones. For a single source in the far field, the TDOA between microphones i and j is related to the source direction by the geometry of the

array. Given TDOAs for multiple pairs, the source direction can be recovered via triangulation or closed-form solutions for specific array layouts [**?**].

Let $x_i(n)$ and $x_j(n)$ denote the discrete-time signals from microphones i and j, with sampling frequency f_s. The cross-correlation between them is

$$R_{ij}(\tau) = \mathbb{E}\left[x_i(n)\, x_j(n+\tau)\right]. \tag{1}$$

In the frequency domain, the cross-power spectral density is

$$G_{ij}(f) = X_i(f)\, \overline{X_j(f)}, \tag{2}$$

where $X_i(f)$ and $X_j(f)$ are the Fourier transforms of x_i and x_j, and $\bar{\cdot}$ denotes complex conjugation. The generalized cross-correlation (GCC) with a weighting function $\Psi(f)$ is

$$R_{ij}^{(\Psi)}(\tau) = \int_{-\infty}^{\infty} \Psi(f)\, G_{ij}(f)\, e^{j2\pi f\tau}\, df. \tag{3}$$

The Phase Transform (PHAT) weighting discards magnitude and retains only phase information:

$$\Psi_{\text{PHAT}}(f) = \frac{1}{|G_{ij}(f)| + \epsilon}, \tag{4}$$

where $\epsilon > 0$ is a small constant to avoid division by zero. This yields

$$R_{ij}^{\text{PHAT}}(\tau) = \int_{-\infty}^{\infty} \frac{G_{ij}(f)}{|G_{ij}(f)| + \epsilon}\, e^{j2\pi f\tau}\, df. \tag{5}$$

The TDOA estimate is the lag that maximizes the GCC-PHAT:

$$\hat{\tau}_{ij} = \frac{1}{f_s} \cdot \arg \max_{\ell \in [-\ell_{\max}, \ell_{\max}]} R_{ij}^{\text{PHAT}}(\ell), \tag{6}$$

where $\ell_{\max} = \lceil \tau_{\max} f_s \rceil$ and $\tau_{\max}$ is the maximum physically feasible delay given the inter-microphone distance. In practice, the integral is computed via the inverse FFT of the weighted cross-spectrum.

For a rectangular four-microphone array, pairs $(1, 2)$ and $(4, 3)$ provide horizontal-axis TDOAs; pairs $(3, 2)$ and $(4, 1)$ provide vertical-axis TDOAs. The normalized delays d_x and d_y are averaged over each axis and converted to an azimuth angle θ via $\theta = \arctan 2(d_y, d_x) - \pi/2$, with appropriate coordinate alignment for the glasses frame.

3.1.2 MUSIC (Multiple Signal Classification)

MUSIC is a subspace-based, high-resolution DoA algorithm [**?**]. It exploits the structure of the spatial covariance matrix to separate signal and noise subspaces.

The received signal vector at frequency f is modeled as

$$\mathbf{x}(f) = \mathbf{A}(\theta)\,\mathbf{s}(f) + \mathbf{n}(f), \tag{7}$$

where $\mathbf{A}(\theta) = [\mathbf{a}(\theta_1), \ldots, \mathbf{a}(\theta_D)]$ is the $M \times D$ steering matrix for D sources, $\mathbf{a}(\theta)$ is the steering vector for direction θ, $\mathbf{s}(f)$ is the $D \times 1$ source vector, and $\mathbf{n}(f)$ is additive noise. For a plane wave from direction θ, the steering vector has elements

$$a_m(\theta) = \exp\left(j\,\frac{2\pi f}{c}\,\mathbf{r}_m^\top \mathbf{u}(\theta)\right), \tag{8}$$

where $\mathbf{r}_m$ is the position of microphone m and $\mathbf{u}(\theta)$ is the unit direction vector.

The spatial covariance matrix is

$$\mathbf{R}_x = \mathbb{E}\left[\mathbf{x}\mathbf{x}^H\right] = \mathbf{A}\mathbf{R}_s\mathbf{A}^H + \sigma_n^2\mathbf{I}, \tag{9}$$

where $\mathbf{R}_s$ is the source covariance matrix and σ_n^2 is the noise power. Eigenvalue decomposition yields

$$\mathbf{R}_x = \mathbf{U}_s\mathbf{\Lambda}_s\mathbf{U}_s^H + \mathbf{U}_n\mathbf{\Lambda}_n\mathbf{U}_n^H. \tag{10}$$

The columns of $\mathbf{U}_s$ span the signal subspace (corresponding to the D largest eigenvalues); the columns of $\mathbf{U}_n$ span the noise subspace. The key property is that steering vectors for true source directions are orthogonal to the noise subspace: $\mathbf{a}^H(\theta)\mathbf{U}_n\mathbf{U}_n^H\mathbf{a}(\theta) = 0$ when θ is a true DoA.

The MUSIC spatial spectrum is

$$P_{\text{MUSIC}}(\theta) = \frac{1}{\mathbf{a}^H(\theta)\,\mathbf{U}_n\mathbf{U}_n^H\,\mathbf{a}(\theta)}. \tag{11}$$

DoA estimates are the D peaks of $P_{\text{MUSIC}}(\theta)$ over the search range. MUSIC requires knowing or estimating D and performs an eigendecomposition per frequency bin; it offers high angular resolution but is sensitive to model mismatch and coherent sources.

3.1.3 SRP-PHAT (Steered Response Power with Phase Transform)

SRP-PHAT is a beamforming-based method that searches over candidate source positions to maximize the output of a steered delay-and-sum beamformer [**?**]. It is robust to reverberation and noise because the PHAT weighting emphasizes phase coherence over magnitude.

For a candidate source position $\mathbf{x}$, the time difference of arrival between microphones m_1 and m_2 is

$$\tau_{m_1,m_2}(\mathbf{x}) = \frac{1}{c}\left(\|\mathbf{x}-\mathbf{p}_{m_1}\| - \|\mathbf{x}-\mathbf{p}_{m_2}\|\right). \tag{12}$$

The steered response power at $\mathbf{x}$ is the sum of generalized cross-correlations evaluated at the corresponding TDOAs:

$$P(\mathbf{x}) = \sum_{m_1=1}^{M}\sum_{m_2=1}^{M} R^{\text{PHAT}}_{m_1,m_2}(\tau_{m_1,m_2}(\mathbf{x})). \tag{13}$$

With PHAT weighting, each $R^{\text{PHAT}}_{m_1,m_2}$ uses only phase information, reducing sensitivity to spectral coloration and reverberation.

The source position estimate is

$$\hat{\mathbf{x}}_s = \arg\max_{\mathbf{x}\in\mathcal{G}} P(\mathbf{x}), \tag{14}$$

where $\mathcal{G}$ is a grid of candidate positions (e.g., on a sphere or circle for 2D azimuth). The DoA is then the direction from the array center to $\hat{\mathbf{x}}_s$. SRP-PHAT avoids eigendecomposition but requires a grid search; resolution is limited by grid spacing and computational cost.

3.1.4 FRIDA (FRI-Based DOA)

FRIDA leverages the finite rate of innovation (FRI) framework to estimate DoA without a grid search [**?**]. It is designed for arbitrary microphone array geometries and coherently combines multi-band information.

The central insight is that for any array layout, entries of the spatial covariance matrix can be linearly transformed into a uniformly sampled sum of sinusoids. The frequencies of these sinusoids encode the DoA. Specifically, for a given frequency bin, the covariance between microphones i and j depends on the phase difference $\phi_{ij}(\theta) = (2\pi f/c)\,(\mathbf{r}_i - \mathbf{r}_j)^\top \mathbf{u}(\theta)$. By arranging

covariance entries in a suitable structure, one obtains a signal of the form $\sum_d \alpha_d e^{j\omega_d n}$, where ω_d is related to the d-th source direction. FRI sampling theory allows recovery of ω_d from a small number of samples, and thus recovery of the DoA, without exhaustive search.

FRIDA achieves state-of-the-art resolution at low signal-to-noise ratios by coherently combining information across frequency bands. It supports both 2D (azimuth only) and 3D (azimuth and elevation) localization and handles multiple sources. The algorithm is implemented in the pyroomacoustics library [**?**].

3.2 DoA Python Prototyping

Before implementing DoA on the embedded microcontroller, the team developed a Python prototyping pipeline to validate algorithms, compare their performance, and generate labeled test data. This pipeline uses PyRoomAcoustics [**?**] to simulate acoustic propagation in a 3D room and produces multi-channel recordings that mimic what the microphone array would capture in real conditions.

3.2.1 Room Simulation with PyRoomAcoustics

PyRoomAcoustics provides a `ShoeBox` room model: a rectangular enclosure with configurable dimensions and acoustic properties. The simulation computes room impulse responses (RIRs) between sources and microphones using the image source method, which models reflections from walls, floor, and ceiling. The received signal at each microphone is the convolution of the source signal with the corresponding RIR, plus optional diffuse noise.

The prototyping code (in `src/prototyping/pyroomacoustics_sim/`) creates a room with dimensions $10 \times 10 \times 3$ metres and sampling frequency $f_s = 8000$ Hz (or 16 kHz in the continuous simulation). Four microphones are placed in a square configuration approximating the glasses form factor:

- Microphone 1: $(4.95, 5.10, 1.75)$ m (front-left)
- Microphone 2: $(5.05, 5.10, 1.75)$ m (front-right)
- Microphone 3: $(4.95, 5.00, 1.75)$ m (back-left)
- Microphone 4: $(5.05, 5.00, 1.75)$ m (back-right)

The array spans approximately 10 cm in each horizontal dimension, consistent with the physical spacing of microphones on the smart glasses.

A sound source is placed at a known position (e.g., $(9, 9, 1)$ m or $(1, 6, 1)$ m). The source signal can be a sine wave, a pre-recorded audio file (e.g., `car_horn_8k.mp3`), or live microphone input. The `room.simulate()` method convolves the source signal with each RIR and produces the multi-channel output.

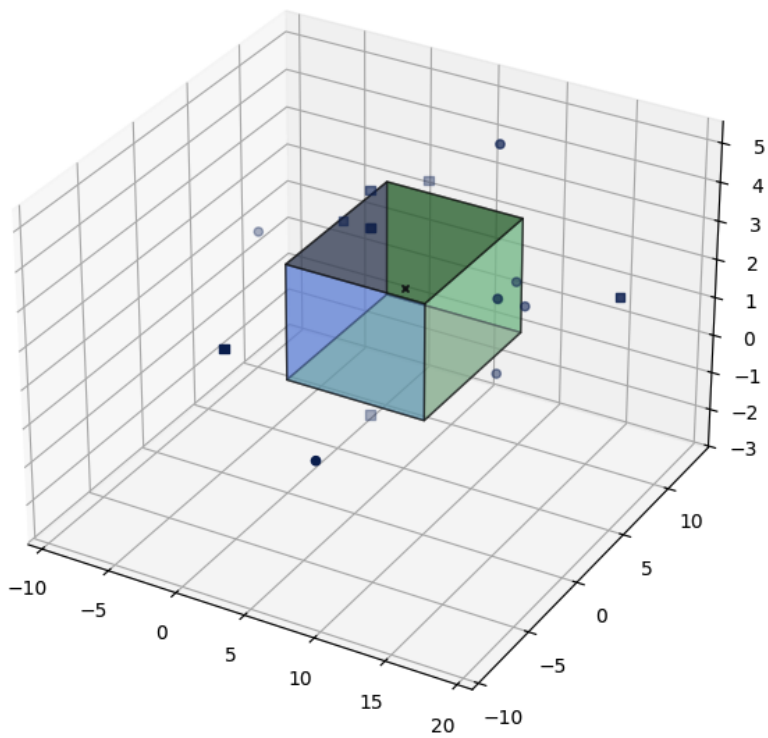

Figure 1: Room layout used in PyRoomAcoustics simulation.

3.2.2 Test Data Generation

After simulation, the multi-channel signals are saved as MP3 files (`mic_1.mp3`, `mic_2.mp3`, `mic_3.mp3`, `mic_4.mp3`) in a directory named for the test case (e.g., `example_mp3_audio_sources`). The `save_room_data()` function in `pyroom_helper.py` normalizes each channel to 16-bit integer range, wraps it in a `pydub` `AudioSegment`, and exports to MP3. A `README.md` is written alongside the audio files, documenting room dimensions, microphone positions, source positions, and sampling frequency. This metadata provides ground-truth DoA for validation.

The workflow is orchestrated by `example2.py`: create room, add microphones, load or generate source signal, add source, simulate, optionally plot the room and microphone responses, and save the data. The resulting files are "accurate" in the sense that they are produced by a physics-based simulation with known source and microphone geometry; the true azimuth can be computed as $\theta_{\text{true}} = \arctan 2(x_s - x_c, y_s - y_c)$ where (x_s, y_s) is the source position and (x_c, y_c) is the array center.

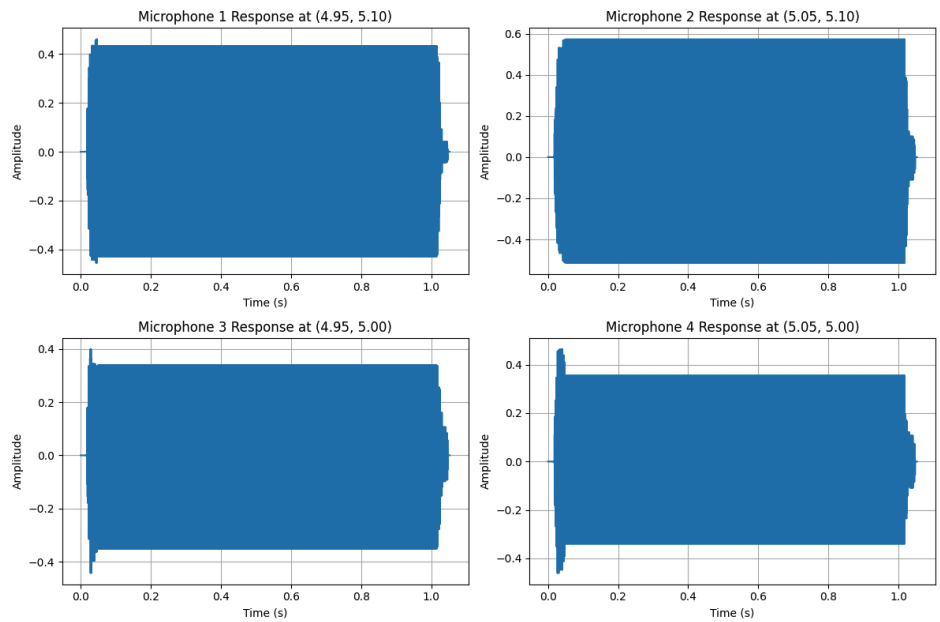

Figure 2: Microphone responses after room simulation.

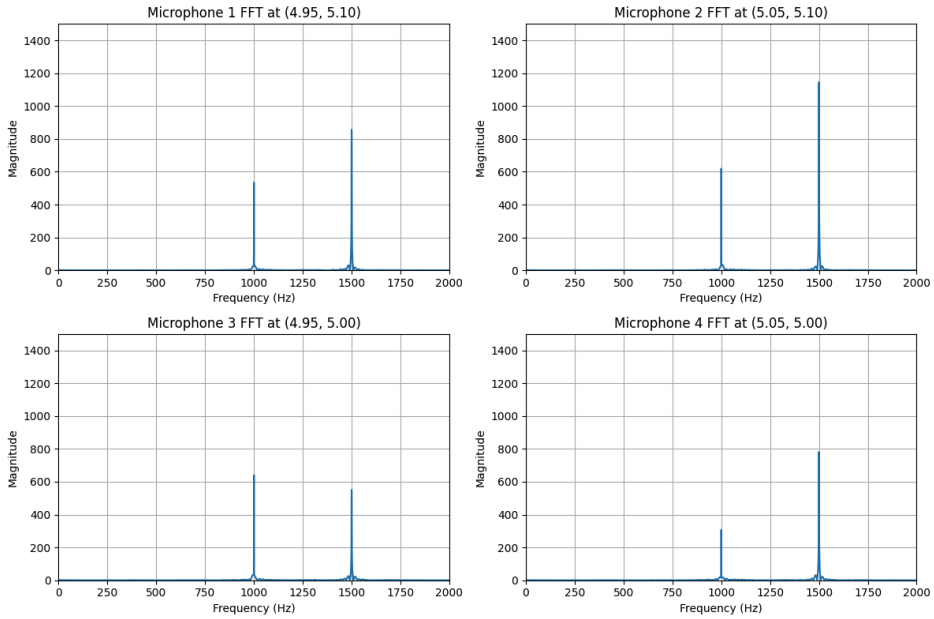

Figure 3: Frequency-domain responses of microphone signals.

3.2.3 Algorithm Testing and Comparison

The `doa_analysis.py` module loads the saved MP3 files, normalizes them to $[-1, 1]$, and converts the time-domain signals to the Short-Time Fourier Transform (STFT) format required by pyroomacoustics DoA algorithms. The STFT uses a Hanning window, FFT size $N_{\text{FFT}} = 256$, and hop size $N_{\text{FFT}}/2$. The frequency range is restricted to $[300, 3000]$ Hz to avoid DC and Nyquist aliasing and to focus on the most informative band for speech and environmental sounds.

Each algorithm is invoked with the same STFT data and microphone array geometry: `pra.doa.MUSIC`, `pra.doa.SRP`, and `pra.doa.FRIDA`. The estimated azimuth angles are stored in `doa_obj.azimuth_recon` (in radians). When a true source position is provided, the true azimuth is computed and compared to the estimates.

The `example_doa.py` script demonstrates the full pipeline: load data from `example_mp3_audio_sources`, run MUSIC, SRP-PHAT, and FRIDA, and print a summary. The `visualize_doa_2d()` function produces a 2D polar plot showing microphone positions, estimated directions (colored arrows), and the true direction (black dashed arrow) when available.

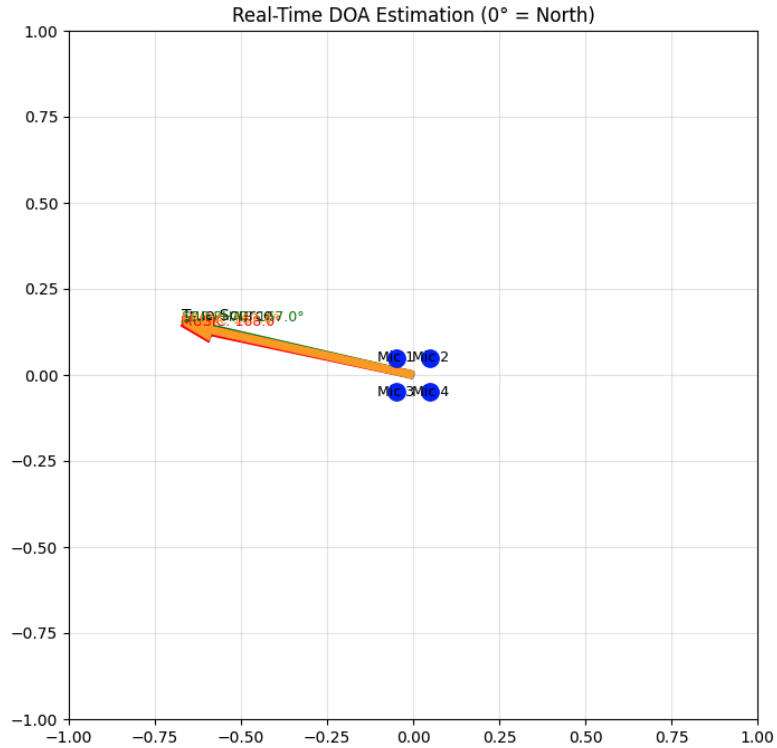

Figure 4: DoA estimation results: comparison of MUSIC, SRP-PHAT, and FRIDA with ground truth.

The continuous simulation (`run_simulation.py`) extends this workflow to live audio: a microphone captures real-time input, which is convolved with precomputed RIRs and fed to the DoA algorithms. The visualization updates continuously as new direction estimates are produced. This mode supports iterative tuning of algorithm parameters and validation under realistic conditions.

4 Classification

[This section is written by the classification partner. —SS]